ASSIGNMENT 11.3

D.Charmi

2303A51314

Batch – 05

Task – 01

Prompt : Create a Python program to store person details (name and mobile) in two ways: one using a list and another using array each supporting insert, find, remove, and display features with a menu interface. If there is no person found, the program should display an appropriate message. Implement both the list and array versions of the program, and ensure that they function correctly.

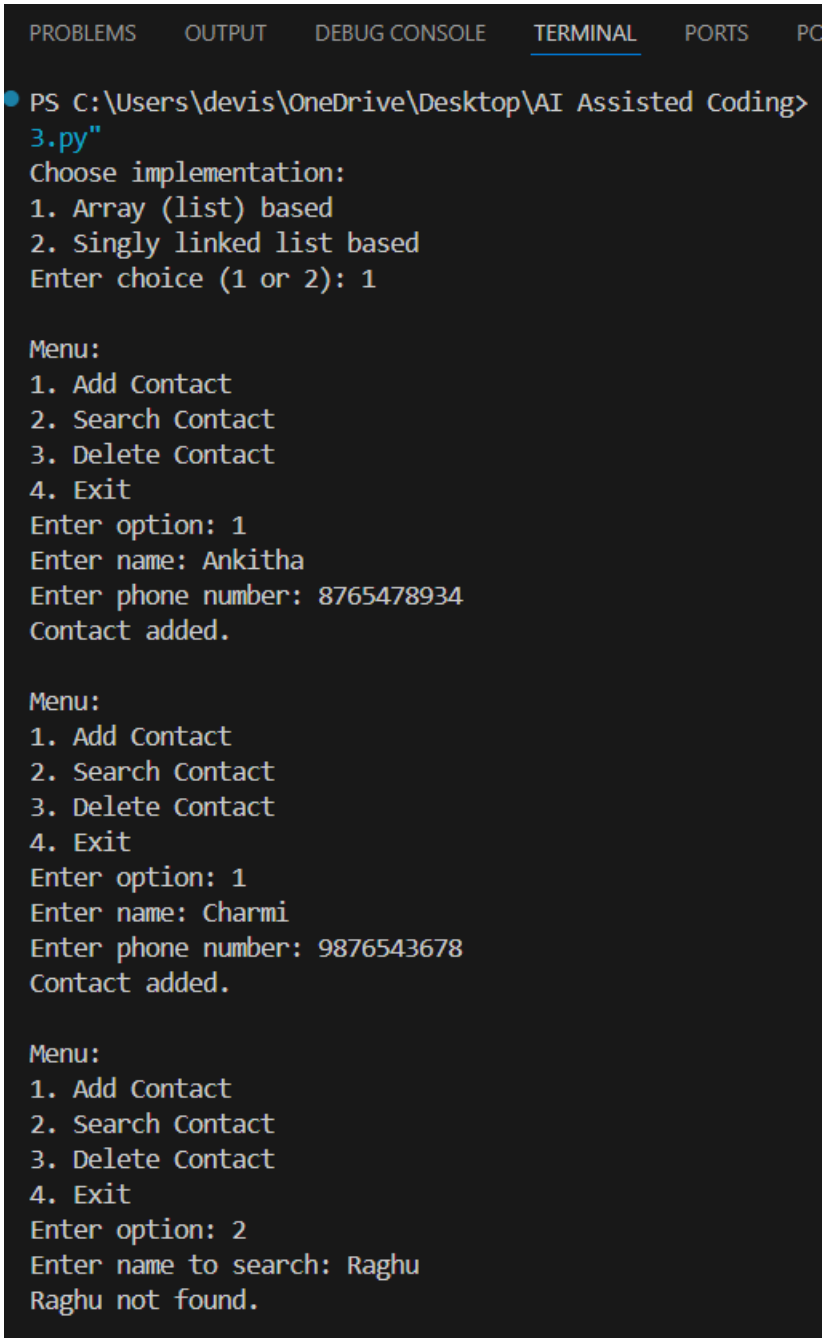
Code:

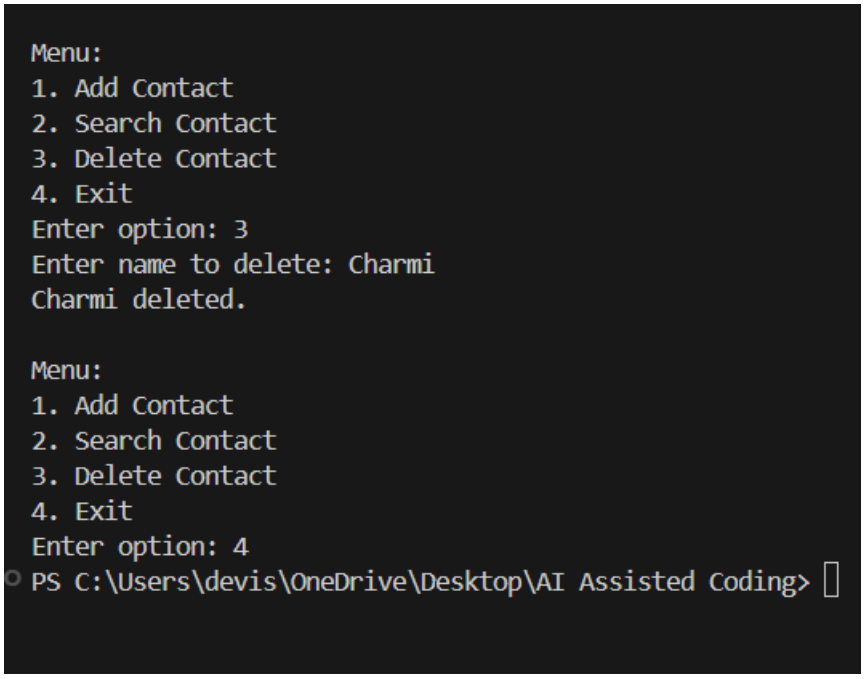
```
#Create a Python program to store person details (name and mobile) in two ways: one using a list and another using array
#each supporting insert, find, remove, and display features with a menu interface.If there is no person found, the program should display an appropriate message.
#Implement both the list and array versions of the program, and ensure that they function correctly.

class Contact:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
class ArrayContactManager:
    def __init__(self):
        self.contacts = []
    def add_contact(self, name, phone):
        contact = Contact(name, phone)
        self.contacts.append(contact)
    def search_contact(self, name):
        for contact in self.contacts:
            if contact.name == name:
                return contact.phone
        return None
    def delete_contact(self, name):
        for i, contact in enumerate(self.contacts):
            if contact.name == name:
                del self.contacts[i]
                return True
        return False
class Node:
```

```
    def __init__(self, contact):
        self.contact = contact
        self.next = None
class LinkedListContactManager:
    def __init__(self):
        self.head = None
    def add_contact(self, name, phone):
        contact = Contact(name, phone)
        new_node = Node(contact)
        new_node.next = self.head
        self.head = new_node
    def search_contact(self, name):
        current = self.head
        while current:
            if current.contact.name == name:
                return current.contact.phone
            current = current.next
        return None
    def delete_contact(self, name):
        current = self.head
        prev = None
        while current:
            if current.contact.name == name:
                if prev:
                    prev.next = current.next
                else:
                    self.head = current.next
                return True
            prev = current
            current = current.next
        return False
def main():
    print("Choose implementation:")
    print("1. Array (list) based")
    print("2. Singly linked list based")
    choice = int(input("Enter choice (1 or 2): "))
    if choice == 1:
        manager = ArrayContactManager()
    elif choice == 2:
        manager = LinkedListContactManager()
    else:
        print("Invalid choice!")
        return
    while True:
        print("\nMenu:")
        print("1. Add Contact")
        print("2. Search Contact")
        print("3. Delete Contact")
```

```
        print("4. Exit")
        option = int(input("Enter option: "))
        if option == 1:
            name = input("Enter name: ")
            phone = input("Enter phone number: ")
            manager.add_contact(name, phone)
            print("Contact added.")
        elif option == 2:
            name = input("Enter name to search: ")
            phone = manager.search_contact(name)
            if phone:
                print(f"Phone number for {name}: {phone}")
            else:
                print(f"{name} not found.")
        elif option == 3:
            name = input("Enter name to delete: ")
            if manager.delete_contact(name):
                print(f"{name} deleted.")
            else:
                print(f"{name} not found.")
        elif option == 4:
            break
        else:
            print("Invalid option!")
if __name__ == "__main__":
    main()
```

Output :


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
3.py"
Choose implementation:
1. Array (list) based
2. Singly linked list based
Enter choice (1 or 2): 1
Menu:
1. Add Contact
2. Search Contact
3. Delete Contact
4. Exit
Enter option: 1
Enter name: Ankitha
Enter phone number: 8765478934
Contact added.
Menu:
1. Add Contact
2. Search Contact
3. Delete Contact
4. Exit
Enter option: 1
Enter name: Charmi
Enter phone number: 9876543678
Contact added.
Menu:
1. Add Contact
2. Search Contact
3. Delete Contact
4. Exit
Enter option: 2
Enter name to search: Raghu
Raghu not found.



Explanation :

The array (list) based version of the contact manager is simpler to implement and provides constant time complexity for adding contacts. However, searching and deleting contacts can take linear time in the worst case, as it may require iterating through the entire list. Memory usage is generally more efficient for small datasets, but it can grow significantly as more contacts are added.

The singly linked list based version allows for more efficient memory usage when adding contacts, as it does not require contiguous memory allocation. However, searching and deleting contacts can also take linear time in the worst case, similar to the array version. The linked list can be more efficient for larger datasets, as it can grow dynamically without needing to resize an underlying array. Overall, the choice between the two implementations depends on the specific use case and requirements of the application.

TASK – 02 :

Prompt : Build a Python program to handle book borrowing requests using two approaches: first with a basic FIFO structure, then with a priority-based structure where faculty entries are served before students, including enqueue and dequeue functions. Test the system with mixed user types and display output showing correct service order in both implementations.

Code :

```python
#Build a Python program to handle book borrowing requests using two approaches: first with a basic FIFO structure, then with a priority-based structure where faculty entries are served before students, including enqueue and dequeue functions.
#Test the system with mixed user types and display output showing correct service order in both implementations.
class FIFOQueue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        self.queue.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.queue.pop(0)
        else:
            return None

    def is_empty(self):
        return len(self.queue) == 0

    def display(self):
        print("FIFO Queue:", self.queue)
class PriorityQueue:
    def     init    (self):
        self.queue = []

    def enqueue(self, item, priority):
        self.queue.append((item, priority))
        self.queue.sort(key=lambda x: x[1])  # Sort by priority

    def dequeue(self):
        if not self.is  empty():
            return self.queue.pop(0)[0]  # Return the item with the highest priority
        else:
            return None

    def is_empty(self):
        return len(self.queue) == 0

    def display(self):
        print("Priority Queue:", [(item, priority) for item, priority in self.queue])
# Testing the FIFO Queue
fifo_queue = FIFOQueue()
fifo_queue.enqueue("Student A")
fifo_queue.enqueue("Faculty B")
fifo  queue.enqueue("Student C")
fifo_queue.display()
```

```
print("Dequeue from FIFO Queue:", fifo_queue.dequeue())
fifo_queue.display()
# Testing the Priority Queue
priority_queue = PriorityQueue()
priority_queue.enqueue("Student A", priority=2)  # Lower number means higher priority
priority_queue.enqueue("Faculty B", priority=1)
priority_queue.enqueue("Student C", priority=2)
priority_queue.display()
print("Dequeue from Priority Queue:", priority_queue.dequeue())
priority_queue.display()
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE

PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:/Users/devis/AppD
3.py"
FIFO Queue: ['Student A', 'Faculty B', 'Student C']
Dequeue from FIFO Queue: Student A
FIFO Queue: ['Faculty B', 'Student C']
Priority Queue: [('Faculty B', 1), ('Student A', 2), ('Student C', 2)]
Dequeue from Priority Queue: Faculty B
Priority Queue: [('Student A', 2), ('Student C', 2)]
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

Explanation :

The FIFOQueue class implements a basic First-In-First-Out structure where items are served in the order they were added. The enqueue method adds items to the end of the queue, while the dequeue method removes and returns the item at the front of the queue. This approach does not differentiate between user types, so all requests are treated equally.

The PriorityQueue class implements a priority-based structure where each item is associated with a priority level. The enqueue method adds items along with their priority and sorts the queue based on priority. The dequeue method removes and returns the item with the highest priority (lowest number). In this implementation, faculty entries are given higher priority than student entries, ensuring that faculty requests are served before student requests.

TASK – 03

Prompt : Create a Python program to manage technical support entries using a LIFO structure with operations to add, remove, view the top element, and check empty/full conditions. Simulate at least five entries to clearly demonstrate reverse-order resolution and include well-structured functions with comments.

Code :

```
#Create a Python program to manage technical support entries using a LIFO structure with operations to add, remove, view the top element, and check empty/full conditions.
#Simulate at least five entries to clearly demonstrate reverse-order resolution and include well-structured functions with comments.
class SupportEntry:
    def __init__(self, issue, customer):
        self.issue = issue
        self.customer = customer
class SupportStack:
    def __init__(self, capacity):
        self.stack = []
        self.capacity = capacity
    def is_full(self):
        return len(self.stack) >= self.capacity
    def is_empty(self):
        return len(self.stack) == 0
    def push(self, entry):
        if not self.is_full():
            self.stack.append(entry)
            print(f"Added support entry: {entry.issue} for {entry.customer}")
        else:
            print("Support stack is full. Cannot add new entry.")
    def pop(self):
        if not self.is_empty():
            entry = self.stack.pop()
            print(f"Removed support entry: {entry.issue} for {entry.customer}")
            return entry
        else:
            print("Support stack is empty. No entries to remove.")
    def peek(self):
        if not self.is  empty():
            entry = self.stack[-1]
            print(f"Top support entry: {entry.issue} for {entry.customer}")
            return entry
        else:
            print("Support stack is empty. No entries to view.")
# Simulating support entries
support  stack = SupportStack(capacity=5)
support_stack.push(SupportEntry("Cannot connect to Wi-Fi", "Alice"))
```

```
support_stack.push(SupportEntry("Printer not working", "Bob"))
support_stack.push(SupportEntry("Software installation issue", "Charlie"))
support_stack.push(SupportEntry("Computer running slow", "David"))
support_stack.push(SupportEntry("Email not syncing", "Eve"))
# Attempting to add another entry to demonstrate full condition
support_stack.push(SupportEntry("Keyboard not responding", "Frank"))
# Viewing the top entry
support_stack.peek()
# Removing entries to demonstrate LIFO order
support_stack.pop()
support_stack.pop()
support_stack.pop()
support_stack.pop()
support_stack.pop()
# Attempting to remove from an empty stack to demonstrate empty condition
support_stack.pop()
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE

PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:/Users/devis/
3.py"
Added support entry: Cannot connect to Wi-Fi for Alice
Added support entry: Printer not working for Bob
Added support entry: Software installation issue for Charlie
Added support entry: Computer running slow for David
Added support entry: Email not syncing for Eve
Support stack is full. Cannot add new entry.
Top support entry: Email not syncing for Eve
Removed support entry: Email not syncing for Eve
Removed support entry: Computer running slow for David
Removed support entry: Software installation issue for Charlie
Removed support entry: Printer not working for Bob
Removed support entry: Cannot connect to Wi-Fi for Alice
Support stack is empty. No entries to remove.
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

Explanation :

This code defines a SupportEntry class to represent individual technical support issues and a SupportStack class to manage these entries using a Last In, First Out (LIFO) structure. The SupportStack class includes methods to check if the stack is full or empty, add new entries (push), remove the most recent entry (pop), and view the top entry (peek). The program simulates adding five support entries, demonstrates the full condition, views the top entry, and removes entries

in reverse order to show how LIFO works. Finally, it attempts to remove an entry from an empty stack to demonstrate the empty condition.

TASK – 04

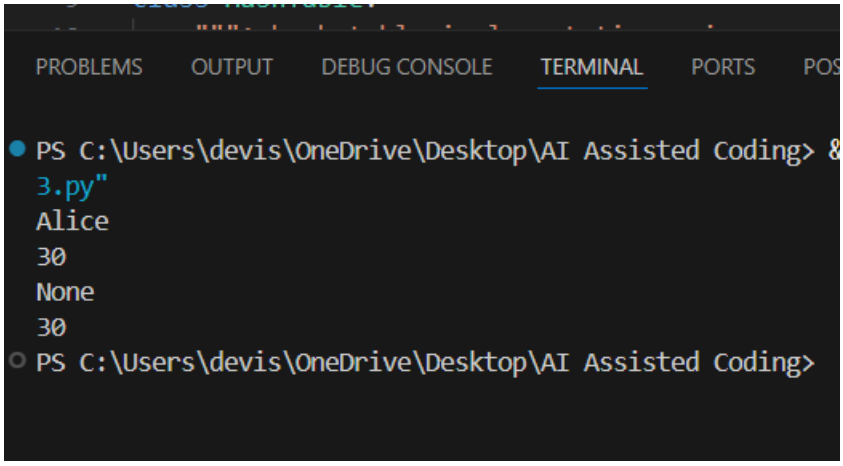
Prompt : Generate a Python class to build a key-value storage system using hashing with separate chaining for collision handling, including methods to add, retrieve, and remove elements. Ensure the implementation is clearly structured, fully commented, and demonstrates how collisions are managed using linked buckets.

Code :

```
#Generate a Python class to build a key-value storage system using hashing with separate chaining for collision handling, including methods to add, retrieve, and remove elements.
#Ensure the implementation is clearly structured, fully commented, and demonstrates how collisions are managed using linked buckets.
class Node:
    """A node in the linked list used for separate chaining."""
    def     init    (self, key, value):
        self.key = key
        self.value = value
        self.next = None
class HashTable:
    """A hash table implementation using separate chaining for collision handling."""
    def     init    (self, size=10):
        self.size = size
        self.table = [None] * self.size
    def _hash(self, key):
        """Generate a hash for the given key."""
        return hash(key) % self.size
    def add(self, key, value):
        """Add a key-value pair to the hash table."""
        index = self.  hash(key)
        new  node = Node(key, value)
        if self.table[index] is None:
            # No collision, simply add the node
            self.table[index] = new  node
        else:
            # Collision handling using separate chaining
            current = self.table[index]
            while current:
                if current.key == key:
                    # Update the value if the key already exists
                    current.value = value
                    return
                if current.next is None:
                    break
```

```
            current = current.next
        # Add the new node at the end of the chain
        current.next = new_node
    def retrieve(self, key):
        """Retrieve the value associated with the given key."""
        index = self._hash(key)
        current = self.table[index]
        while current:
            if current.key == key:
                return current.value  # Key found, return value
            current = current.next  # Move to the next node in the chain
        return None  # Key not found
    def remove(self, key):
        """Remove the key-value pair associated with the given key."""
        index = self._hash(key)
        current = self.table[index]
        prev = None
        while current:
            if current.key == key:
                if prev is None:
                    # Removing the first node in the chain
                    self.table[index] = current.next
                else:
                    # Bypass the node to
                    prev.next = current.next
                return True  # Key found and removed
            prev = current
            current = current.next
        return False  # Key not found
# Example usage:
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.add("name", "Alice")
    hash_table.add("age", 30)
    print(hash_table.retrieve("name"))  # Output: Alice
    print(hash_table.retrieve("age"))  # Output: 30
    hash_table.remove("name")
    print(hash_table.retrieve("name"))  # Output: None
    print(hash_table.retrieve("age"))  # Output: 30
```

Output :



Explanation :

This code defines a hash table using separate chaining to handle collisions. The `Node` class represents each entry in the chain, while the `HashTable` class manages the overall structure. The `_hash` method computes the index for a given key, and the `add`, `retrieve`, and `remove` methods allow for adding, retrieving, and removing key-value pairs. When a collision occurs (i.e., multiple keys hash to the same index), the code uses a linked list (chain) to store all entries at that index, allowing for efficient handling of collisions.

TASK – 05

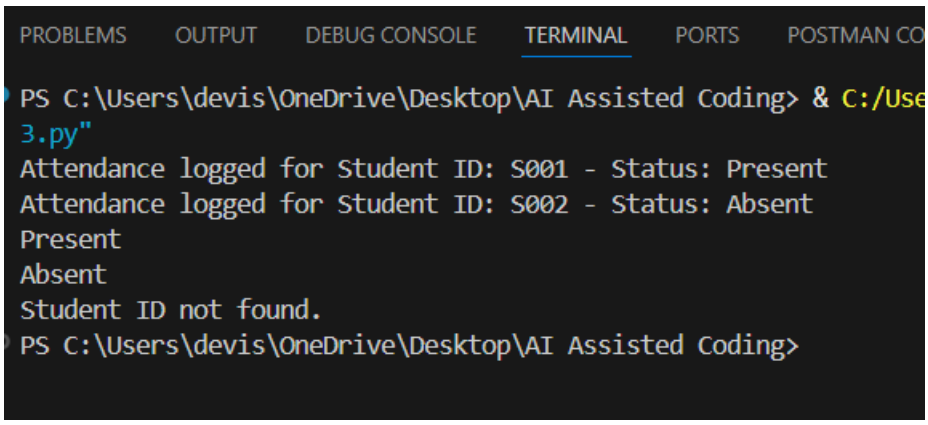
Prompt : Design a Python-based campus management application covering attendance logging, event sign-ups, book issuing, transport timing coordination, and food counter handling; choose suitable data structures for each feature and justify each choice briefly in 2–3 sentences. Create a mapping table (Feature → Structure Used → Reason), implement one chosen feature with well-commented Python code

Code :

```
#Design a Python-based campus management application covering attendance logging, event sign-ups, book issuing, transport timing coordination, and food counter handling; choose suitable data structures for each feature and justify each choice briefly in 2–3 sentences.
#Create a mapping table (Feature → Structure Used → Reason), implement one chosen feature with well-commented Python code

# Implementing the Attendance Logging Feature
class CampusManagement:
    def __init__(self):
        # Initialize an empty dictionary to store attendance records
        self.attendance_log = {}

    def log_attendance(self, student_id, status):
```

```
        """
        Log attendance for a student.

        Parameters:
        student_id (str): The unique identifier for the student.
        status (str): The attendance status (e.g., 'Present', 'Absent').

        This method updates the attendance_log dictionary with the student's attendance status.
        """
        self.attendance_log[student_id] = status
        print(f"Attendance logged for Student ID: {student_id} - Status: {status}")

    def check_attendance(self, student_id):
        """
        Check the attendance status of a student.

        Parameters:
        student_id (str): The unique identifier for the student.

        Returns:
        str: The attendance status of the student or a message if the student ID is not found.

        This method retrieves the attendance status from the attendance_log dictionary using the student ID as
the key.
        """
        return self.attendance_log.get(student_id, "Student ID not found.")
# Example usage
campus = CampusManagement()
campus.log  attendance("S001", "Present")
campus.log_attendance("S002", "Absent")
print(campus.check  attendance("S001"))  # Output: Present
print(campus.check  attendance("S002"))  # Output: Absent
print(campus.check  attendance("S003"))  # Output: Student ID not found.
```

Output :



Explanation :

Feature → Structure Used → Reason

Attendance Logging → Dictionary → A dictionary allows for efficient storage and retrieval of attendance records using student IDs as keys, providing O(1) time complexity for lookups.

Event Sign-ups → List → A list can be used to store event sign-ups as it allows for easy appending of new entries and iteration over the list to display all sign-ups.

Book Issuing → Dictionary → A dictionary can store book information with book IDs as keys, allowing for quick access to book details and tracking issued books efficiently.

Transport Timing Coordination → List of Tuples → A list of tuples can store transport schedules, where each tuple contains the transport type and its timing, allowing for easy iteration and display of schedules.

Food Counter Handling → Queue → A queue can manage food orders in a first-come, first-served manner, ensuring that orders are processed in the order they were received.